SERVICE ORIENTED ARCHITECTURES

ACIT3855

AGENDA

- Quick Review
- Quiz 5
- Asynchronous Messaging
- Lab 6
  - Setup Kafka and MySQL on VM
  - Integrate with your Receiver and Storage services

QUICK REVIEW

- What's the difference between Synchronous and Asynchronous communication?
- What are the benefits of using messaging for communication between Microservices?
- What are some of the use cases of Apache Kafka?
- What are the names of the applications involved in a system using Apache Kafka? Where are the messages stored?
- What are some of the features of Apache Kafka?

QUIZ 5

- Quiz is on the Learning Hub
- Open book, do your own work
- You have <15 minutes to complete it

COURSE SCHEDULE

Week	Topics	Notes
1	• Services Based Architecture Overview • RESTful APIs Review	Lab 1
2	• Microservices Overview • Edge Service	Lab 2, Quiz 1
3	• Database Per Service • Storage Service (SQLite)	Lab 3, Quiz 2
4	• Logging, Debugging and Configuration • Storage Service (MySQL)	Lab 4, Quiz 3
5	• RESTful API Specification (OpenAPI) • Processing Service	Lab 5, Quiz 4
6	• Synchronous vs Asynchronous Communication • Message Broker Setup and Messaging	Lab 6, Quiz 5
7	• Event Sourcing • Analysis Service	Lab 7, Quiz 6
8	**Midterm Exam**	
9	• Deployment Part 1 - Containerization (Docker/Docker Compose)	Lab 8, Quiz 7
10	• Deployment Part 2 – Containerization (Docker Volumes and Bind Mounts)	Lab 9, Quiz 8
11	**Computing Reading Week**	
12	• Dashboard UI and CORS	Lab 10, Quiz 9
13	• Deployment Part 3 – Provisioning • Issues and Technical Debt	Lab 11, Quiz 10
14	• Deployment Part 4 – Load Balancing and Scaling • Final Exam Preview	Lab 12, Quiz 11
15	• Review Week	Lab 12 Demos, Assignment Due
16	**Final Exam**	

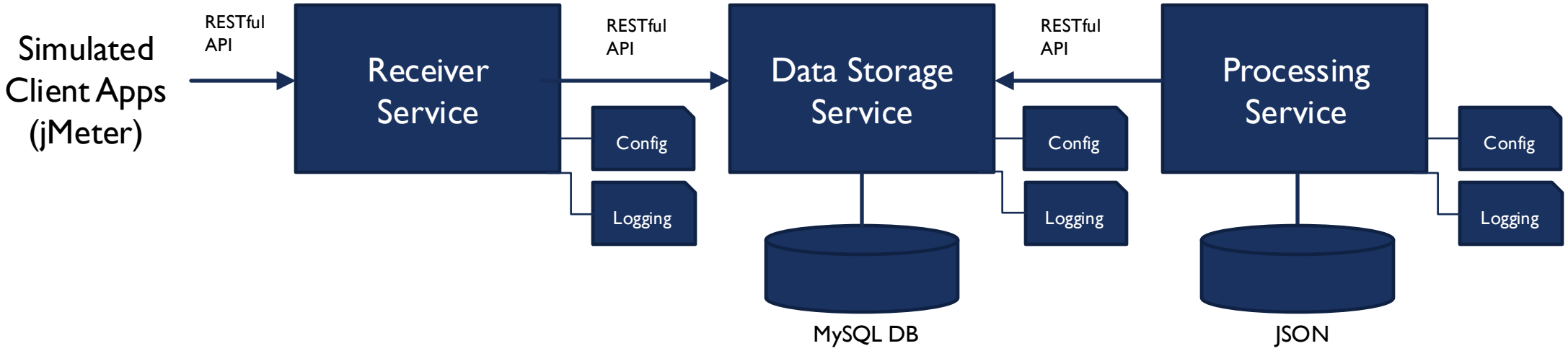
OUR SAMPLE APPLICATION

Our sample application will have three initial services:

- Receiver Service (Lab 2)
- Storage Service (Lab 3)
- Processing Service (Lab 5)

We will upgrade our services to enable scalability and availability:

- Database Service – MySQL (Lab 4)
- Event Based Messaging – Kafka (Labs 6 and 7)



SYNCHRONOUS VS ASYNCHRONOUS COMMUNICATION

Synchronous Communication – When a client makes a request and has to wait until the request fulfilled before receiving the response. The client thread generally is blocked until the server fulfills the request. The server thread has to complete all the activities required of the request before returning the response.

An HTTP call to a RESTful service is considered synchronous communication.

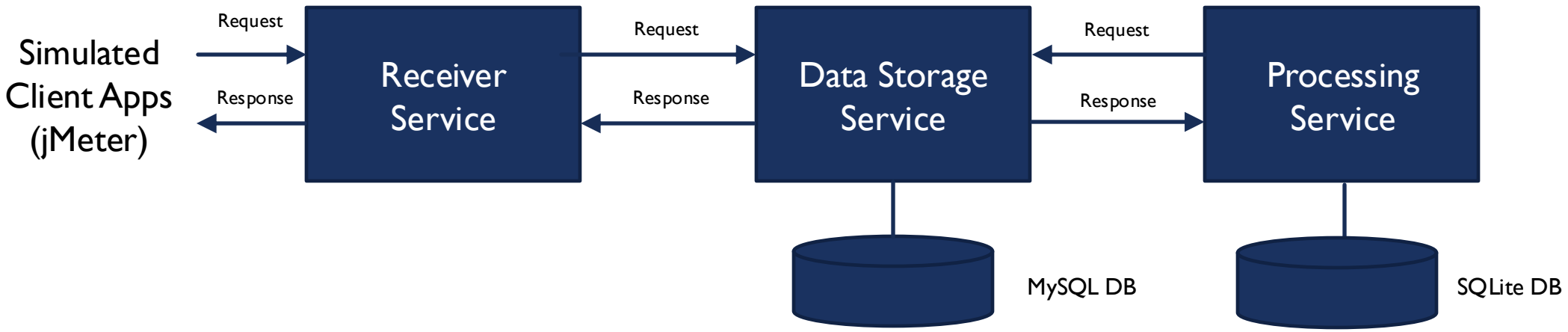
Asynchronous Communication – When a client makes a request and DOES NOT have to wait until the request completely fulfilled before receiving the response.

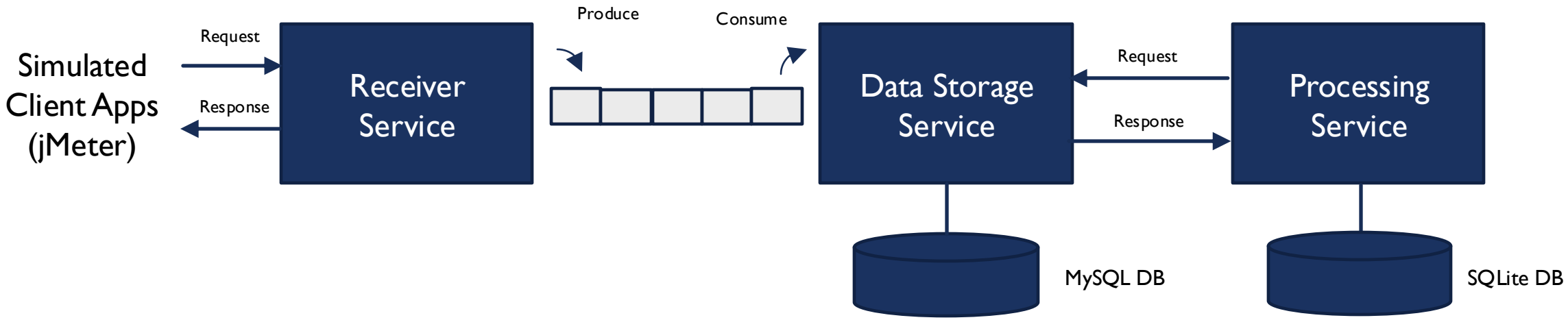
BENEFITS OF ASYNCHRONOUS MESSAGING (FROM THE READING)

- **Simple, scalable connectivity**
- **Simple, high availability**
- **Simple producer/consumer scalability**
- **The enablement of publish/subscribe, message filtering, routing and fanout**
- **Message rate and consumer availability decoupling**

Source: https://solace.com/blog/messaging-between-microservices/

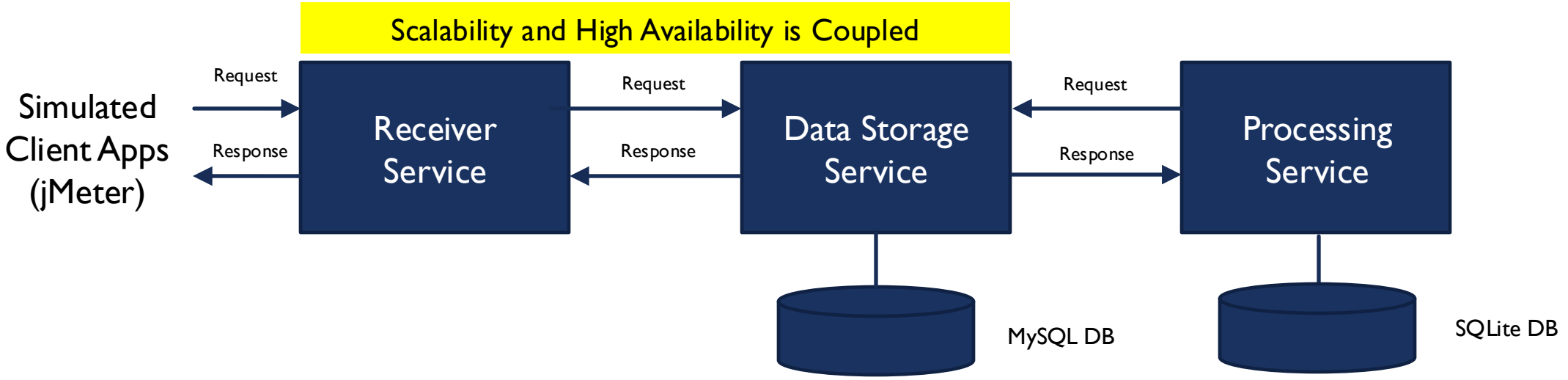
SYNCHRONOUS VS ASYNCHRONOUS COMMUNICATION

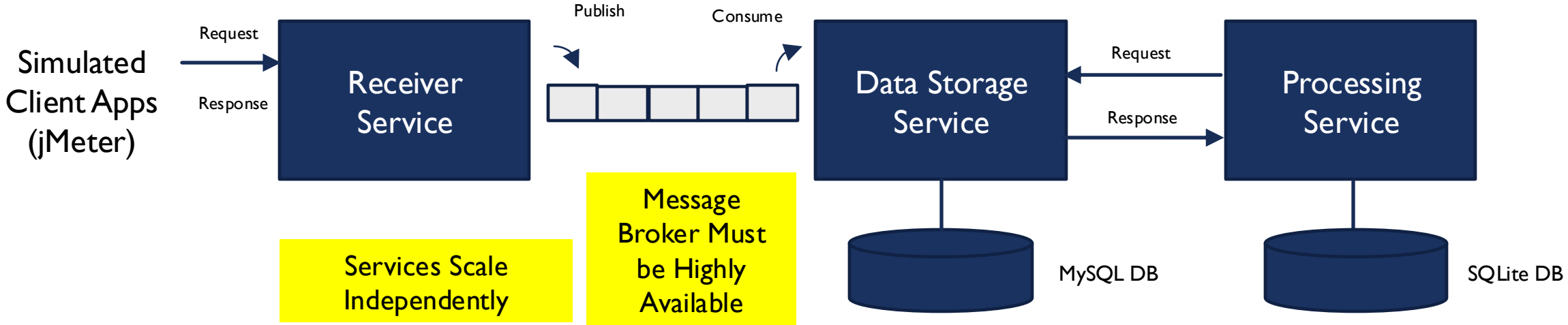




SYNCHRONOUS VS ASYNCHRONOUS COMMUNICATION

AVAILABILITY AND SCALABILITY





ASYNCHRONOUS COMMUNICATION - MESSAGE BROKERS

- A **Message Broker** is a software application that allows messages to be routed from one or more senders to one or more receivers. May also include validation and transformation of the messages.
- The senders are typically called Producers (or Publishers).
- The receivers are typically called Consumers (or Subscribers)
- A Message Broker may support one or more **Message Queues**. A Message Queue is a logical construct that transports specific types or groups of messages from the Producers to the Consumers.
- Producers publish messages to a Message Queue. Consumers subscribe to a Message Queue to receive messages.
- Examples:

APACHE KAFKA VS RABBITMQ

Tool	Apache Kafka	RabbitMQ
Message Ordering	Provides message ordering because of its partitions. Messages are sent to topics by message key.	Not supported.
Message Lifetime	Kafka is a log, which means that messages are always there. You can manage this by specifying a message retention policy.	RabbitMQ is a queue, so messages are done away with once consumed, and acknowledgment is provided.
Delivery Guarantees	Retains order only inside a partition. In a partition, Kafka guarantees that the whole batch of messages either fails or passes.	Doesn't guarantee atomicity, even in relation to transactions involving a single queue.
Message Priorities	N/A	In RabbitMQ, you can specify message priorities, and consumed message with high priority at the onset.

Source: https://www.upsolver.com/blog/kafka-versus-rabbitmq-architecture-performance-use-case

Apache Kafka Use Cases

- Streams with complex routing, throughput of 100K/sec events or more, with "at least once" partitioned ordering
- Applications requiring a stream history, delivered in "at least once" partitioned ordering. Clients can see a 'replay' of the event stream.
- Event sourcing, modeling changes to a system as a sequence of events.
- Stream processing data in multi-stage pipelines. The pipelines generate graphs of real-time data flows.
- Note: A Topic in Kafka is equivalent to a Message Queue. Topics can be further subdivided into Partitions.

RabbitMQ Use Cases

- Applications that need to support legacy protocols, such as STOMP, MQTT, AMQP, 0-9-1.
- Granular control over consistency/set of guarantees on a per-message basis
- Complex routing to consumers
- Applications that need a variety of publish/subscribe, point-to-point request/reply messaging capabilities.

MESSAGING

Microservices Principle:

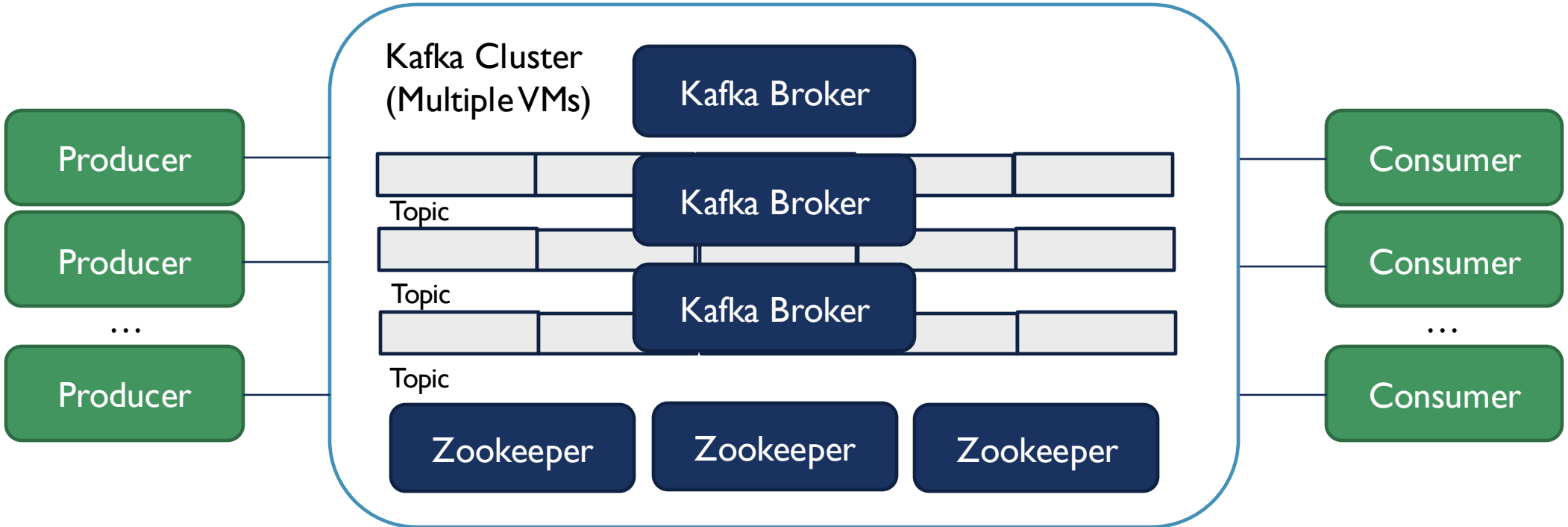
- Dumb Pipes and Smart Endpoints (more toward Kafka)
  - Pipe (i.e., Message Broker) is merely a highly-available transport mechanism (minimal logic)
  - Endpoints (i.e., Services) have the logic on how or if to process the messages

Traditional SOA: Smart Pipes and Dumb Endpoints (more toward RabbitMQ)

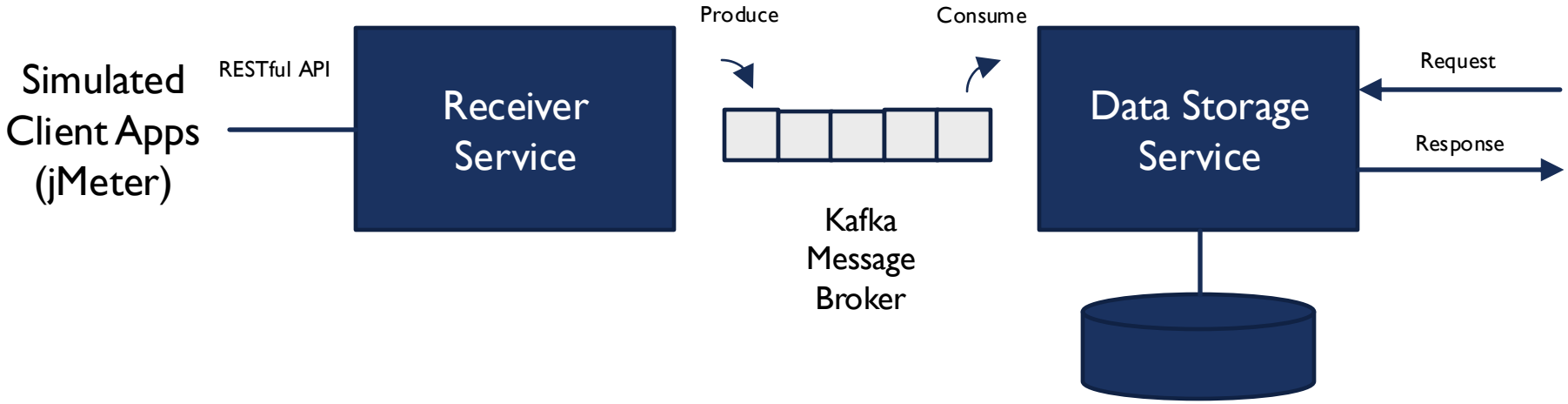
- Pipe (i.e., Message Broker) has complex logic – i.e., routing, retry
- Endpoints (i.e., Services) just process the messages they are given

KAFKA DEPLOYMENT

- Kafka is typically deployed in a Cluster
- There are multiple Kafka Brokers to allow for redundancy and high-availability (minimum 3 brokers for HA)
- There are multiple Zookeeper configuration stores that manage the state and configuration of Kafka (minimum of 3 Zookeepers for HA)



DATA STORAGE SERVICE - COMPLEXITY



When we integrate the Data Storage Service with the Message Broker that means it has two key tasks:

- Consuming Messages
- Handling Requests on the RESTful API

Why might this be a challenge? How might we solve it?